Introducción a Anaconda: instalación y gestión de entornos

¿Qué es Anaconda?

Anaconda es una distribución de Python (y R) orientada a ciencia de datos y computación científica. Incluye más de 300 librerías preinstaladas y, sobre todo, el gestor de paquetes y entornos **conda**.

Para el trabajo del día a día en proyectos como este, lo más habitual es usar **Miniconda**: una versión mínima de Anaconda que solo instala Python y conda, y a partir de ahí instalas únicamente lo que necesitas.

Anaconda vs Miniconda

	Anaconda	Miniconda
Tamaño	~3 GB	~100 MB
Incluye	Python + 300 librerías + GUI	Solo Python + conda
Recomendado para	Principiantes, todo listo	Proyectos concretos, control total

Para este curso usaremos **Miniconda** (o Anaconda si ya lo tienes instalado, funciona igual).

Instalación de Miniconda

Windows

1. Descarga el instalador desde docs.conda.io/en/latest/miniconda.html — elige la versión **64-bit** para Python 3.11 o superior.
2. Ejecuta el `.exe` y sigue el asistente. Opciones recomendadas:
   - Instalar solo para el usuario actual (no requiere administrador).
   - **No** marcar "Add Anaconda to my PATH" (usa el *Anaconda Prompt* en su lugar).
   - Sí marcar "Register Anaconda as my default Python".
3. Abre **Anaconda Prompt** desde el menú de inicio para usar conda.

macOS

```
# Descarga e instala con el script de shell
curl -O https://repo.anaconda.com/miniconda/Miniconda3-latest-MacOSX-x86_64.sh
bash Miniconda3-latest-MacOSX-x86_64.sh
```

Para Apple Silicon (M1/M2/M3), usa la versión `arm64`:

```
curl -O https://repo.anaconda.com/miniconda/Miniconda3-latest-MacOSX-arm64.sh
bash Miniconda3-latest-MacOSX-arm64.sh
```

Sigue las instrucciones del instalador y reinicia la terminal.

Linux

```
curl -O https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
```

Acepta la licencia, confirma la ruta de instalación (por defecto `~/miniconda3`) y permite que el instalador inicialice conda en tu shell.

Verificar la instalación

```
conda --version
# conda 24.x.x
```

Conceptos clave

Canal (channel)

Los paquetes en conda se distribuyen a través de **canales** (repositorios). Los más importantes son:

- `defaults` — canal oficial de Anaconda.
- `conda-forge` — canal comunitario, más actualizado y con más paquetes. **Recomendado para geoespacial.**

Entorno (environment)

Un **entorno** es una instalación de Python aislada con sus propias librerías y versiones. Permite tener proyectos con dependencias incompatibles sin que interfieran entre sí.

```
base     ← entorno base (no trabajar aquí)
lidar    ← entorno de este curso
otro     ← otro entorno independiente
```

> **Buena práctica:** nunca instales paquetes en el entorno `base`. Crea siempre un entorno específico para cada proyecto.

Gestión de entornos

Crear un entorno

```
# Entorno con una versión específica de Python
conda create -n mi_entorno python=3.11

# Entorno desde un fichero environment.yml
conda env create -f environment.yml
```

Activar y desactivar

```
conda activate lidar    # activar
conda deactivate        # volver al entorno base
```

Una vez activado, verás el nombre del entorno al principio del prompt:

```
(lidar) usuario@equipo:~$
```

Listar entornos disponibles

```
conda env list
# o equivalente:
conda info --envs
```

Eliminar un entorno

```
conda env remove -n nombre_entorno
```

Instalar y gestionar paquetes

```
# Instalar un paquete (siempre con el entorno activo)
conda install numpy

# Especificar canal
conda install -c conda-forge geopandas

# Instalar varios paquetes a la vez
conda install numpy pandas matplotlib

# Actualizar un paquete
conda update geopandas

# Eliminar un paquete
conda remove shapely
```

```
# Listar paquetes instalados en el entorno activo
conda list
```

---

conda vs pip dentro de un entorno conda

Dentro de un entorno conda puedes usar tanto `conda install` como `pip install`. Ambos instalan paquetes en el entorno activo, pero funcionan de forma muy diferente.

Resolución de dependencias

- **conda** resuelve dependencias a nivel de sistema también: librerías C, GDAL, PROJ, GEOS… Cuando instalas `geopandas` con conda, te instala automáticamente GDAL compilado y todo lo que necesita.
- **pip** solo resuelve dependencias Python. Si un paquete necesita una librería C externa, asume que ya está en el sistema.

Esto es crítico en geoespacial: instalar `rasterio` o `fiona` con pip en Windows o macOS suele dar problemas porque necesitan GDAL compilado. Con conda simplemente funciona.

Conocimiento mutuo

- **conda sabe lo que instaló pip**, pero no al revés. Si mezclas los dos, pip puede romper dependencias que conda había resuelto sin que conda se entere.
- Por eso, si usas un `environment.yml`, el orden importa: primero todo lo que venga de conda, y al final el bloque `pip:` solo para lo que no esté en ningún canal conda.

Cuándo usar cada uno

Situación	Usa
Paquete disponible en conda-forge	`conda install -c conda-forge paquete`
Librerías geoespaciales (GDAL, rasterio, fiona…)	Siempre conda
Paquete solo en PyPI	`pip install paquete`
Paquetes muy nuevos o experimentales	pip (suelen llegar antes a PyPI)

Regla práctica

Usa conda siempre que puedas, y pip solo como último recurso para lo que no esté en conda-forge. Nunca mezcles los dos para el mismo paquete.

Entorno del curso: `lidar`

Instalación paso a paso

```
conda create -n lidar python=3.11
conda activate lidar
conda install -c conda-forge pdal python-pdal proj-data laspy lazrs-python \
              rasterio geopandas shapely matplotlib numpy jupyterlab
```

> **`proj-data`** es imprescindible: instala el geoide oficial español `es_ign_egm08-rednap.tif`, necesario para convertir alturas elipsoidales (WGS84, lo que exporta DJI Terra) a alturas ortométricas (lo que usa el PNOA).

Verificar el geoide

Una vez instalado el entorno, verifica que PROJ encuentra el geoide oficial:

```
projinfo -s EPSG:4979 -t EPSG:25830+5782
```

En la Operación nº 1 del resultado debe aparecer:

```
+step +inv +proj=vgridshift +grids=es_ign_egm08-rednap.tif +multiplier=1
```

Si aparece `at least one grid missing` en todas las operaciones, reinstala `proj-data`:

```
conda install -c conda-forge proj-data --force-reinstall
```

Alternativa: instalar desde `environment.yml`

Si tienes el fichero `environment.yml` del repositorio:

```
conda env create -f environment.yml
conda activate lidar
```

Actualizar el entorno si cambia el fichero

```
conda env update -f environment.yml --prune
```

El flag `--prune` elimina los paquetes que ya no están en el fichero.

El fichero `environment.yml`

El fichero `environment.yml` permite reproducir un entorno exacto en cualquier máquina. Es la forma recomendada de compartir entornos en proyectos colaborativos.

El entorno de este curso tiene la siguiente estructura:

```yaml
name: lidar
channels:
  - conda-forge
```

```
  - defaults
dependencies:
  - python=3.11
  - pdal
  - python-pdal
  - proj-data
  - laspy
  - lazrs-python
  - rasterio
  - geopandas
  - shapely
  - matplotlib
  - numpy
  - jupyterlab
```

Exportar tu entorno actual

```
# Exportación completa (incluye versiones exactas, útil para reproducibilidad)
conda env export > environment.yml

# Exportación solo de lo instalado explícitamente (más portable entre plataformas)
conda env export --from-history > environment.yml
```

Usar el entorno con Jupyter

Cuando lanzas JupyterLab, los notebooks necesitan un **kernel** — el proceso Python que ejecuta el código. Por defecto, Jupyter usa el kernel del entorno desde el que se lanza. Para que tus entornos conda aparezcan como kernels disponibles hay dos formas:

Opción 1 — `nb_conda_kernels` (recomendada)

Instala `nb_conda_kernels` en el entorno **base** una sola vez:

```
conda activate base
conda install -c conda-forge nb_conda_kernels
```

A partir de ahí, **cualquier entorno** que tenga `ipykernel` instalado aparecerá automáticamente como kernel disponible en JupyterLab, sin tener que hacer nada más. Es la opción más cómoda si trabajas con varios entornos.

Opción 2 — Registrar manualmente el entorno

Desde el entorno que quieres registrar:

```
conda activate lidar
python -m ipykernel install --user --name lidar --display-name "Python (lidar)"
```

Esto registra ese entorno concreto como kernel disponible para el usuario.

¿Cuál usar?

	`nb_conda_kernels`	Registro manual
Instalación	Una vez en `base`	Una vez por entorno
Nuevos entornos	Aparecen solos	Hay que registrarlos
Modifica `base`	Sí (mínimamente)	No

> **Importante:** el kernel seleccionado en el notebook (esquina superior derecha en JupyterLab) es lo que determina qué librerías están disponibles. Asegúrate siempre de tener seleccionado el kernel **`lidar`** antes de ejecutar las celdas.

Lanzar JupyterLab

```
conda activate lidar
jupyter lab
```

Se abrirá el navegador automáticamente en `http://localhost:8888`. Si no, copia la URL que aparece en la terminal.

Referencia rápida

Acción	Comando
Crear entorno del curso	`conda create -n lidar python=3.11`
Crear entorno desde fichero	`conda env create -f environment.yml`
Activar entorno	`conda activate lidar`
Desactivar entorno	`conda deactivate`
Listar entornos	`conda env list`
Instalar paquete	`conda install -c conda-forge paquete`
Actualizar paquete	`conda update paquete`
Listar paquetes	`conda list`
Actualizar entorno desde fichero	`conda env update -f environment.yml --prune`
Eliminar entorno	`conda env remove -n nombre`
Lanzar JupyterLab	`jupyter lab`
Ver info de conda	`conda info`